

The Value of Personalization in Large Language Model Responses

Andrey Fradkin¹, Yan (Stella) Si², Giorgos Zervas^{1,2}

¹Questrom School of Business, Boston University

²Faculty of Computing & Data Sciences, Boston University

September 2026

Abstract

We study heterogeneous preferences for LLM responses and how personalized routing across models increases end-user utility. To do so, we estimate a random-coefficients demand model on Arena preference data and simulate the effects of a variety of model routers. A router which uses just the prompt raises predicted mean per-user utility over the best single model, attaining a 53.8% model-implied head-to-head win rate. Allowing the router to personalize to each user’s own taste vector, rather than serving a model catered to the average user’s pick, raises the win rate to 71.0%. The allocation of responses includes all models we consider, even though some have much higher benchmark scores than others. We then investigate the feasibility of learning individual preferences using repeat interactions. After forty feedback rounds, we can close 72% of the gap to the individual taste-based router. We also consider the efficacy of verbalizing estimated tastes. This approach yields a 70.4% predicted preference for the intended user versus a non-verbalized prompt. Lastly, we evaluate routers that choose a model before generation, and find these to be less effective. Even after forty past choices, this ex-ante router recovers just 35% of the ex-post gain.

1 Introduction

A common view of progress in AI is that scaling training data, compute, and inference is sufficient to achieve arbitrarily good performance across a variety of benchmarks. An implicit assumption behind this belief is that there is a primary dimension of intelligence that explains most of the value of the model. Yet LLM responses differ across a variety of dimensions including style, length, tone, format, and other dimensions not captured by benchmark intelligence. If consumers differ in their preferences for these characteristics across different tasks, then no single model will be best for everyone across tasks. In this paper, we document large gains in utility from using the prompt and user’s preferences to

route requests across models. To do so, we propose an empirical framework for measuring the gains from a variety of routing approaches.

We begin by estimating a random-coefficients discrete-choice model in which utility depends on the judged accuracy of a response, model identity and capabilities, response style, the prompt, and a user’s latent tastes. The estimation uses preference data from Arena, a platform that presents users with anonymized responses from two randomly assigned models and asks which they prefer (Chiang et al. 2024). We score every response for factual accuracy by an LLM judges. Accuracy is the biggest driver of preference, though with diminishing returns. How much accuracy matters depends on the prompt, and users differ widely in how much they value a partly correct answer. Style preferences are also important for preference. We estimate heterogeneous preferences for a variety of characteristics (e.g., tables, response length) and interactions between these characteristics and prompt embeddings. These heterogeneous preferences affect which model is best across users. If each user must be assigned a single model, 56% have a personal optimum different from the mean-taste winner. No model dominates the ten-model candidate set: under personalized assignment, the leading model accounts for only 25.8% of user-prompt choices, and the top two models together receive 43.8%, leaving more than half to eight other models.

Heterogenous preferences generate large gains from routing. Relative to always using the best single model in our sample, a prompt-level router raises mean predicted utility and attains a 53.8% model-implied head-to-head win rate. Giving the router knowledge of each user’s taste vector further increases utility gains and the win rate to 71.0%. Personalization (vs knowing just the prompt) accounts for about three quarters of the total utility gain. This benefit comes from frequently selecting models that have lower benchmark intelligence on average than the prompt-only router’s selections.

Next, we consider whether the personalized routing performance can be approached in practice by learning preferences through repeated elicitation of pairwise preferences. We simulate 40 rounds of pairwise feedback, and use a Bayesian approach to learn the preference vector. This approach closes 72% of the utility gap between an unpersonalized router and a router that knows each user’s preference vector. This exercise is closely related to recent methods that represent users by latent variables or ideal points in a shared preference space (Poddar et al. 2024; Chen et al. 2025), as well as Shenfeld et al. (2025), who factorize user-specific rewards into shared base functions and use active learning to infer a new user’s weights from a small number of pairwise comparisons.

We also test a simpler intervention that requires no model fine-tuning but does require an estimate of preferences: a natural-language preamble that instructs the model to match a user’s estimated tastes. The resulting responses are preferred by the intended user over the unsteered response with predicted probability 70.4%, and the same holds for every model

we steer. Measured against the same baseline on the same prompts and users, steering alone attains a 69.2% win rate, about as high as the user-preference-level router on the same nine-model menu (70.1%) and far above the prompt router (52.7%), and steering the best model for each user and prompt attains an 80.0% win rate.

These gains do not come for free. Our main router operates ex-post: it generates one response from each of ten models and then selects among them, increasing inference costs. To address this concern, we evaluate ex-ante routers that choose a model using only information available before generation, thus saving the cost of prompting all ten models before routing. We benchmark these against the same baseline, always using the best model in the sample, on held-out prompts and users, where the ex-post personalized router attains a 70.7% win rate and the ex-post prompt router 53.8%. In one approach, we predict each model’s response characteristics, including its accuracy bin, from the prompt embedding and score them through the choice model. This prompt-only ex-ante router attains a win rate of only 50.3%, essentially a coin flip against the baseline: response characteristics are hard to predict before generation.

In another approach, we built a router that uses a taste vector learned from 40 of the user’s past pairwise choices to pick one best model without generating responses from all models; it attains a 55.3% win rate, 24% of the gain of the ex-post user-preference-level router. Combining the two approaches raises the win rate to 57.2%, 35% of that gain. A replicated sampling audit shows why prompt-only prediction fails: on a fixed prompt, re-sampling the same model moves its utility by nearly as much as the prompt-specific edge that a router exploits.

The closest paper to ours is Caoui (2025), who uses LMArena choices to estimate demand for differentiated LLMs and decomposes changes in consumer surplus into improvements in intelligence and other model attributes. We use a similar demand framework but shift the counterfactual from measuring market-wide quality growth to assigning responses across models and users. Our contribution is to quantify separately the value of prompt-level routing and user-level personalization in a choice model that holds response-level accuracy fixed, and to evaluate two mechanisms for learning and acting on individual tastes. The paper also complements work documenting pluralistic preferences in human feedback (Kirk et al. 2024) and divergence between aggregate and individual LLM rankings (Garbacea, Wang, and Tan 2026). Our findings are also consistent with Demirer, Fradkin, and Tadelis (2026), who document horizontal and vertical differentiation in the market for LLM inference and show that no single model dominates across use cases. More broadly, our results provide empirical content for models in which user heterogeneity and customization shape the product design and pricing of LLM services (Bergemann, Bonatti, and Smolin 2025).

The paper proceeds as follows. Section 2 describes the data, the response attributes, the

accuracy judge, and the choice model. Section 3 reports the estimates. Section 4 measures the value of prompt routing and personalization and the feasibility of learning tastes from feedback. Section 5 personalizes a single model with language. Section 6 evaluates ex-ante routers, and Section 7 audits response sampling noise. Section 8 discusses limitations and implications, and Section 9 concludes. The appendices give the data construction, the full coefficient tables and identification diagnostics, and the details of every simulation.

2 Data and empirical framework

Arena, originally introduced as Chatbot Arena and also known as LMArena, is a widely used online platform for evaluating large language models through human preference comparisons. Its public model rankings are frequently cited by leading model developers and companies (Chiang et al. 2024). Users submit their own prompts and receive side-by-side responses from two randomly assigned models, whose identities remain hidden until after voting. Users can select either response, declare a tie, or judge both responses to be bad. These comparisons, called “battles,” are aggregated into a public leaderboard. The platform therefore collects evaluations of responses to user-chosen tasks, encompassing both substantive quality and presentation. We use the underlying battle-level choices to estimate preferences over model and response characteristics.

2.1 Arena choices and response attributes

We pool two public releases, covering June–August 2024 and April–July 2025.¹ The English corpus has 128,850 battles (257,700 responses); after dropping incomplete observations and the 18 battles in which a response could not be judged for accuracy, 128,366 battles remain. The seeded 90/10 split contains 115,440 training and 12,926 test battles. The outcomes are A win (33.6%), B win (34.5%), tie (17.2%), and both bad (14.7%). We randomize ordinary ties between A and B using a fixed seed and treat “both bad” as an outside option.

Each response has thirteen features: token length; counts of ordered lists, unordered lists, headers, and bold spans; indicators for tables and emoji; Flesch reading ease and the SMOG index; positive and negative sentiment; and assertive and tentative hedging. Deterministic parsers construct all but the final four indicators, which `openai/gpt-oss-20b`, called through OpenRouter at temperature zero, classifies. Continuous features are standardized using training-sample moments. Prompts are embedded with OpenAI’s `text-embedding-3-large` and reduced to three partial-least-squares (PLS) scores, allowing the value of an attribute to change with the request; the projection is fit on training battles only and applied to held-out and simulation prompts. Appendix A provides the full construction.

¹The releases are `arena-human-preference-100k` and `arena-human-preference-140k`.

Table 1: Estimation data. Counts are battles; outcome shares use the 128,850-battle English corpus before the complete-feature and judged-accuracy restrictions.

Sample	Battles	Outcome	Share
English corpus	128,850	A wins	33.6%
Complete estimation sample	128,366	B wins	34.5%
Training split	115,440	Tie	17.2%
Held-out split	12,926	Both bad	14.7%

Each model has three attributes that are constant across prompts: a benchmark intelligence index, context window, and a reasoning indicator. We measure intelligence with the Artificial Analysis Intelligence Index, a composite of benchmark scores reported on a nominal 0–100 scale, and enter it in units of ten points. We use this index because it covers all 106 models in the estimation sample.² The context window is measured in millions of tokens, and the reasoning indicator equals one for models that use extended chain-of-thought reasoning before responding.

2.2 Judged accuracy

We also create a factual-accuracy score for every response, using a model (GLM 5.2) that is sufficiently more capable than the models in the Arena dataset. The model sees the prompt and response, and returns a score from 0 to 10 with a short rationale; As a robustness check, we also scored a subsample of responses using a Claude Opus 4.8 judge. The two judges’ scores correlate at 0.80 and induce the same within-battle ordering in 95% of the 21 battles in which both are decisive. Scores enter the preference model below as falling into one of three bins: low accuracy (0–3; 14.8% of responses), mid accuracy (4–6; 18.0%), and high accuracy (7–10; 67.2%). The binning lets the value of accuracy be nonlinear and avoids treating the judge’s scale as cardinal. The same judge scores every response used downstream: the ten-model routing panel, the replicated sampling audit, and the verbally personalized generations, so no counterfactual is ever scored without the accuracy term. Appendix B gives the judge details.

2.3 Choice model

Let i index users, p index prompts, and m index language models. In each battle, a user compares two responses to the same prompt and either selects one or rejects both. The

²The Epoch Capabilities Index predicts judged accuracy (Section 2.2) somewhat better at the model level (R^2 of 0.70 against 0.57) but is available for only 73 of them.

systematic utility from model m 's response to prompt p is

$$V_{ipm} = \left(\gamma_0 + \Gamma h_p + \text{diag}(\sigma^A) \nu_i^A \right)' d_{pm} + \left(\beta_0^R + B^R h_p + \text{diag}(\sigma^R) \nu_i^R \right)' x_{pm}^R \\ + \left(B^M h_p + \text{diag}(\sigma^M) \nu_i^M \right)' x_m^M + \delta_m + \alpha \mathbf{1}\{\text{position B}\}, \quad (1)$$

where the superscripts A , R , and M distinguish accuracy, response, and model characteristics. The vector d_{pm} contains the two accuracy-bin indicators (mid and high, with low as the reference). The vector x_{pm}^R contains the thirteen response attributes described above, including length, formatting, readability, sentiment, and hedging. The vector x_m^M contains three model characteristics: the benchmark intelligence index, context window, and reasoning indicator. These are constant across prompts for a given model. The vector h_p contains the three PLS scores constructed from the prompt embedding.

The coefficient vector β_0^R gives mean preferences for response attributes at $h_p = 0$. The matrices B^R and B^M , of dimensions 13×3 and 3×3 , respectively, shift mean preferences with the prompt: $\beta_0^R + B^R h_p$ is the population mean response-coefficient vector conditional on prompt p , while $B^M h_p$ captures prompt-dependent shifts in preferences for model characteristics. Baseline differences across models enter through δ_m ; the specification does not include a separate vector of common mean coefficients on x_m^M .

Unobserved preference heterogeneity enters through $\nu_i = ((\nu_i^A)', (\nu_i^R)', (\nu_i^M)')' \sim N(0, I_{18})$, where ν_i^A , ν_i^R , and ν_i^M contain two, thirteen, and three independent standard-normal taste components. The vectors σ^A , σ^R , and σ^M give the standard deviations of the random coefficients, and $\text{diag}(\sigma)$ denotes a diagonal matrix with σ on its diagonal. For a given prompt, a larger element of σ implies greater dispersion across users in the corresponding marginal utility. The same taste vector applies to both responses in a battle. Because individuals are not tracked across battles, we estimate population heterogeneity rather than preferences for identifiable users.

The fixed effect δ_m captures persistent differences in utility across models after accounting for judged accuracy, observed response attributes, and prompt interactions. There are 74 fixed effects: one for each named model from the seven largest providers and a pooled effect for all other models. Finally, $\mathbf{1}\{\text{position B}\}$ equals one if the response appears in position B (on the right of the screen when a user submits preferences) and zero otherwise; α is position effect coefficient.

To obtain choice probabilities, let $j \in \{A, B, 0\}$ index the two displayed responses and the outside option, where $j = 0$ denotes a ‘‘both bad’’ vote. Total utility is $U_{ipj} = V_{ipj} + \varepsilon_{ipj}$, with V_{ipA} and V_{ipB} given by Equation (1) for the respective models. We normalize systematic outside-option utility to $V_{ip0} = 0$ and assume that the ε_{ipj} are independent standard type-I extreme-value shocks, independent of ν_i . These assumptions fix the location and scale of utility, which we therefore measure in logits: a utility advantage of Δ implies a head-to-

head win probability of $\text{logit}^{-1}(\Delta)$, so advantages of 0.5, 1, and 2 logits correspond to win probabilities of about 62%, 73%, and 88%. Because the advantage varies across user-prompt cells, the model-implied win rates we report are averages of $\text{logit}^{-1}(\Delta U)$ and do not map one-to-one to the mean gain in logits. Integrating the conditional logit probability over the taste distribution gives

$$P_{ip}(j) = \int \frac{\exp\{V_{ipj}(\nu)\}}{1 + \exp\{V_{ipA}(\nu)\} + \exp\{V_{ipB}(\nu)\}} \phi_{18}(\nu) d\nu, \quad (2)$$

where ϕ_{18} is the standard multivariate-normal density in eighteen dimensions; the numerator is one for the outside option. We estimate the model’s 162 parameters by simulated maximum likelihood, approximating the integral with 256 Sobol/antithetic draws (McFadden and Train 2000). Every downstream number is computed from Equation (1) through one shared scoring path, so the routers, the learning simulation, the verbal experiment, and the sampling audit are scored identically by construction.

2.4 Identification

Identification of user-level dispersion comes from how the observed choice probability changes as response-feature gaps vary across battles. Because a person is not tracked longitudinally, the normal mixing distribution is a population model, not a claim that we have recovered any observed user’s vector. To assess parameter recovery, we generate synthetic choices from the model using the reported estimates as the true parameter vector (80,000 battles, 80 models, 100 replications) and re-estimate it. Mean response coefficients, the accuracy gains, and fixed effects recover closely (fixed-effect correlation 0.99 in every replication, accuracy gains within 2%), and the large spreads that carry the heterogeneity interpretation (length 1.07, emoji 1.43, table 1.45, tentative hedging 0.70) are reproduced within 3% on average. True taste spreads below roughly 0.1 are not recovered: they return at the 0.02 optimization floor in 40 to 54% of replications and inflated by a multiple in the rest, and the positive-tone spread (0.41) is understated by 13%. This pattern motivates our focus on length, table, emoji, and hedging heterogeneity.

3 Preference estimates

This section reports estimates of the preference model in equation (1). We first describe the value of judged accuracy, then the distribution of style preferences conditional on accuracy, and finally the model fixed effects.

Table 2: Selected estimates from the mixed logit: the accuracy gains and the four style features with the largest user-level spreads. Accuracy bins are low (0–3, reference), mid (4–6), and high (7–10); style tastes are per unit of the standardized feature (length) or per indicator. Hessian standard errors are in parentheses (delta method for spreads). Model fit is reported in Appendix Table 10.

Parameter	Mean	User-level spread
Accuracy: mid vs. low	1.100 (0.026)	1.592 (0.068)
Accuracy: high vs. low	1.761 (0.026)	0.027 (0.081)
Style: response length (per SD)	0.491 (0.028)	1.066 (0.048)
Style: table indicator	0.450 (0.053)	1.448 (0.178)
Style: emoji indicator	0.196 (0.030)	1.432 (0.108)
Style: tentative hedging	−0.132 (0.020)	0.699 (0.087)

3.1 The value of accuracy

Table 2 reports the main estimates. Among the response characteristics, judged accuracy has by far the largest effect on choice. Relative to a low-accuracy response, a mid-accuracy response raises utility by 1.100 logits and a high-accuracy response by 1.761 logits, compared with 0.491 logits for a one-standard-deviation increase in length. Allowing the accuracy gains to vary with the prompt and across users improves held-out fit relative to a specification with common accuracy coefficients (likelihood-ratio statistic of 1,473 on eight degrees of freedom; Appendix Table 10).

The returns to accuracy are concave. The incremental gain from mid to high accuracy is $1.761 - 1.100 = 0.661$ logits, so moving a response from the low to the mid bin is worth about 1.7 times as much as moving it from the mid to the high bin. The raw data display the same pattern at a smaller magnitude. Among the 70,208 battles with a decided or randomized outcome in which the two judged scores differ, the higher-scored response wins 57.3 percent of the time; among the 35,632 such battles in which the responses fall in different bins, it wins 63.1 percent of the time. The structural estimates are larger because they hold model identity and response style fixed. The concavity also suggests that the judge’s 0–10 scale should not be treated as cardinal.

The value of accuracy varies substantially across prompts. The interactions of the first prompt score with the high- and mid-accuracy indicators are 5.26 (standard error 0.16) and 3.27 (0.20), respectively, so the gain from a correct answer is several times larger at one end of the first PLS axis than at the other. The gains also vary across users, but asymmetrically. The user-level standard deviation of the mid-bin gain is 1.59, against a mean of 1.10, which implies that a nontrivial share of users value a partly correct answer more than a fully correct one. The standard deviation of the high-bin gain, by contrast, is at the optimization floor. Users therefore disagree about the value of a partly correct answer but agree on the value of a fully correct one.

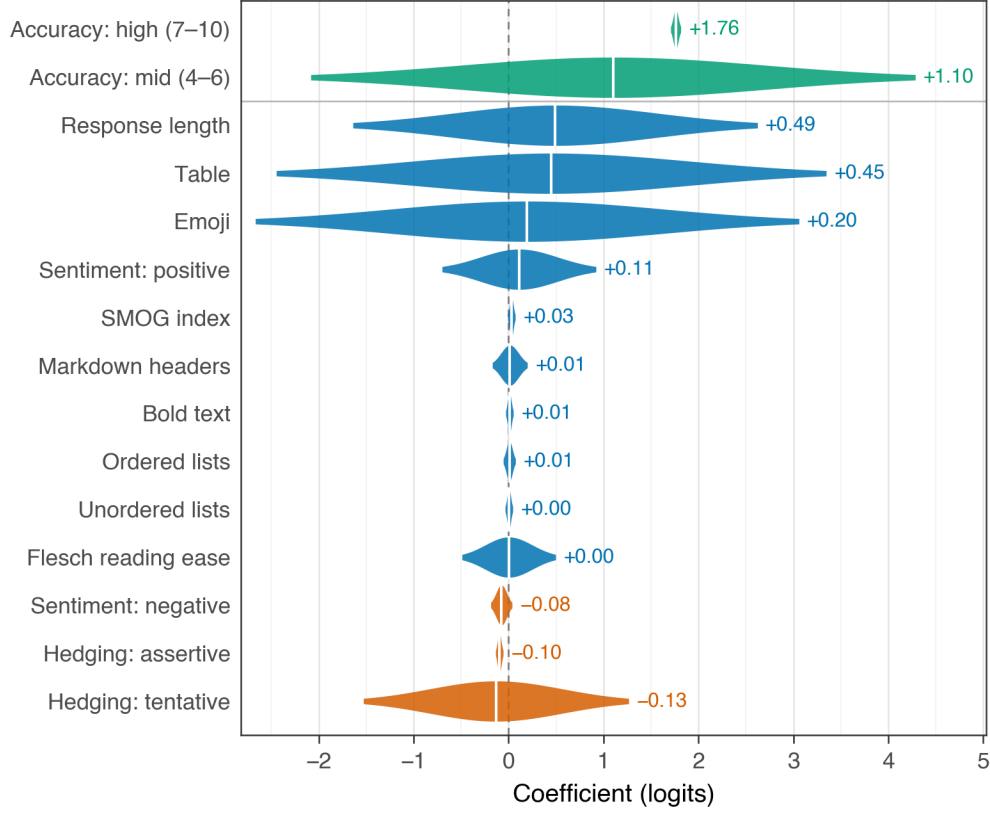


Figure 1: Estimated distribution across users of the accuracy gains and style coefficients, in logits; accuracy bins are relative to a low (0–3) response and style coefficients are per standardized unit. Each shape is a normal density with the estimated mean and user-level standard deviation, drawn to ± 2 standard deviations; the white line and printed value mark the population mean. Features whose standard deviation sits at the optimization floor appear as thin spikes.

A simpler specification yields a consistent result. In a pairwise logit of decided battles on the difference in benchmark intelligence alone, ten index points are worth 0.215 logits. Adding the other model attributes, the thirteen style differences, and the two accuracy-bin differences reduces the intelligence coefficient by 22.6 percent, to 0.166, while the accuracy-bin coefficients are 0.83 and 1.16 (Appendix C). Part of the association between benchmark scores and user choice therefore operates through realized accuracy and style rather than through an intrinsic model premium.

3.2 Style preferences conditional on accuracy

Holding judged accuracy fixed, style preferences remain economically important and heterogeneous (Figure 1). The mean coefficient on the table indicator is 0.450, with a user-level standard deviation of 1.448; the corresponding figures are 0.196 and 1.432 for

emoji and 0.491 and 1.066 for standardized length. Tentative hedging has a negative mean coefficient (-0.132) but a standard deviation of 0.699, and positive tone has a mean of 0.110 and a standard deviation of 0.405. For list counts, bold text, and assertive hedging, the estimated standard deviations are at or near the optimization floor. Heterogeneity is thus concentrated in a few dimensions, and along these dimensions a substantial fraction of users have coefficients of the opposite sign to the population mean. Vertical and horizontal differentiation coexist: once a response is reasonably accurate, additional accuracy is worth less and differences in style account for a larger share of the variation in utility.

Style preferences also vary with the prompt. The interactions of length with the second and third prompt scores are large and precisely estimated (-3.40 and -1.82), so the same length is valued very differently across requests. The table interactions (-1.26 , -1.57 , and 1.91) imply that a table raises utility on some prompts and lowers it on others. Appendix C reports the full set of interactions.

3.3 Model fixed effects

The model fixed effects are the other major component of utility. They capture persistent differences across models in capability, brand, default behavior, and any omitted response attribute correlated with model identity, net of judged accuracy and observed style (Appendix Figure 5 plots all 74). A second-stage OLS regression of the 73 named fixed effects on model attributes associates a ten-point increase in the benchmark index with a 0.160-logit increase in the fixed effect ($t = 5.3$). This association should not be interpreted causally, because benchmark scores are correlated with unobserved model characteristics. Models with higher benchmark scores may, for example, also receive more post-training aimed at conversational quality, or may be better at tailoring their responses to the prompt. We therefore interpret the second-stage coefficients as descriptive of the sources of persistent differences across models. When we repeat the second stage on the fixed effects from two standard logits (Appendix C), the coefficient on the benchmark index falls by about half once response characteristics are included. This is consistent with more capable models being rewarded in part for producing more accurate and better-formatted responses.

4 The value of personalized routing

4.1 Policies and evaluation

The response panel contains 4,916 topic-stratified prompts with complete generations from ten models: GPT-4.1, GPT-4o, Claude Sonnet 4, Gemini 2.5 Pro, DeepSeek V3, Llama 4 Maverick, Grok 4, Qwen3 235B, Kimi K2, and MiniMax M1. Appendix D gives the exact API identifiers. Each prompt-model cell has one response, scored for style and judged for accuracy. We compare:

Table 3: Model-scored welfare relative to always using Gemini 2.5 Pro. The final row is the incremental gain of the user-preference-level router over the prompt router.

Policy	Mean ΔU	Implied win rate
Prompt router	0.305	53.8%
User-preference-level router	1.321	71.0%
Personalization: user-preference level – prompt	1.016	67.2%

1. *Fixed*: always serve Gemini 2.5 Pro, the best single model in the panel.
2. *Prompt router*: serve $\arg \max_m V_{0pm}$, setting $\nu_i = 0$.
3. *User-preference-level router*: serve $\arg \max_m V_{ipm}$ using the user’s true simulated ν_i .
The router knows the user’s taste vector but not the logit shocks ε_{ipj} , so it maximizes expected rather than realized utility and is not an upper bound on the utility of the user’s ex-post preferred response.

The counterfactual samples 1,000 users from $N(0, I)$ with a fixed seed. For policy r , utility gain is $\Delta U_{ipr} = V_{ip,r(p,i)} - V_{ip,\text{Gemini}}$. We report mean gain and the average logit probability $\text{logit}^{-1}(\Delta U)$ that the policy’s response wins a pairwise comparison. These are predictions of the estimated utility model; no new human votes are collected.

4.2 Prompt routing and the value of user-level personalization

The prompt router raises mean predicted utility by 0.305 logits and reaches a 53.8% model-implied win rate against Gemini 2.5 Pro (Table 3). The gain is modest because the baseline router serves Gemini itself on 53.6% of prompts, where the gain is zero by construction, and gains are positive in 28.3% of user-prompt cells. The user-preference-level router adds another 1.016 logits, about 3.3 times the prompt-routing gain, and raises the total gain to 1.321. Because the prompt router’s choice is in the choice set of the user-preference-level router, the incremental gain is nonnegative by construction; it is strictly positive in 65.5% of cells. Personalization is therefore about three quarters of the total gain from routing.

The assignments of the user-preference-level router show that the ten models are not vertically ordered from the perspective of all users. When users may switch models across prompts, Gemini 2.5 Pro and Grok 4 together account for 43.8 percent of user-prompt cells, the leading model receives about one quarter, and the remaining eight models account for more than half (Table 4). GPT-4o, a widely used model, is chosen in only 2.5 percent of cells. These shares are counterfactual assignments under the estimated taste distribution rather than market shares, and they abstract from prices, defaults, access, and switching costs.

A distinct question is which single model maximizes each user’s utility averaged over prompts. In this one-model-per-user assignment, Gemini 2.5 Pro is the best model for 44

Table 4: Assignment shares of the user-preference-level router over user-prompt cells. Users may switch models across prompts.

Model	Share	Model	Share
Gemini 2.5 Pro	25.8%	MiniMax M1	6.2%
Grok 4	18.0%	Claude Sonnet 4	5.8%
DeepSeek V3	12.5%	Kimi K2	4.6%
Qwen3 235B	11.0%	Llama 4 Maverick	2.8%
GPT-4.1	10.7%	GPT-4o	2.5%

percent of users and Grok 4 for 16 percent; DeepSeek V3, GPT-4.1, and Qwen3 235B are each best for about 12 percent, and six models exceed 2 percent. For 56 percent of users, the best model differs from the one that is best at the mean taste. The per-user and per-cell shares answer different questions and should not be compared directly: a single model can be the most common personal choice without being optimal for every user, and allowing the assignment to vary across prompts as well disperses it much further. A similar pattern holds across topics. The population pick share of the leading model, Gemini 2.5 Pro, is 17.4 percent overall, and it leads in five of seven topics; Qwen3 235B leads in mathematics and Grok 4 in multimedia. The leader is statistically distinguishable from the runner-up in every topic except multimedia (Appendix D). The gains of the user-preference-level router therefore do not reflect a relabeling of one universally superior model.

The measured gains mix model fixed effects, judged accuracy, style, prompt interactions, and taste matching. A decomposition of the 0.30 prompt-router gain assigns -0.13 logits to fixed effects, 0.12 to judged accuracy, 0.27 to mean style, and 0.04 to prompt interactions: the router gives up fixed-effect utility relative to Gemini and more than recovers it through accuracy and style. The user-preference-level router adds 1.02 from taste matching, part of which is each user’s own taste for a partly right answer. Only part of the fixed-effect component is statistically associated with benchmark intelligence, so converting the total into benchmark points is not meaningful. The direct capability index of selected models makes the same point: it averages 67.9 for the prompt router but falls to 62.5 under the user-preference-level router, which trades vertical quality for taste fit.

The two personalization margins operate differently. Prompt routing exploits predictable changes in average tastes, Bh_p and Γh_p , as well as realized response features. It can send a coding prompt and a writing prompt to different models without knowing who submitted either request. User personalization instead changes the coefficient vector applied to the same candidate set. Its value is greatest when models span the attributes on which σ is large and when a user’s taste draw is far from zero. The gain from user-level personalization would shrink if suppliers converged to the same response style, even if their average capability stayed unchanged.

Ablations support this interpretation but also reveal the importance of persistent model

quality. Setting all model fixed effects to zero changes 48.0% of mean-user prompt choices on the 4,916-prompt panel. Setting the entire block of mean response-style coefficients to zero changes 52.4% and concentrates nearly all assignments on xAI and Google. Removing the accuracy gains changes 10.9%: the candidate models are close in judged accuracy on most prompts, so accuracy separates responses less often than style does within this menu. Among individual style terms, length changes 22.0% of choices when its mean coefficient is removed; tables have a comparable mean coefficient but affect fewer choices because the feature is binary and rare. These exercises are descriptive perturbations of the fitted objective, not causal estimates of how a provider would change its response if a feature were mandated.

The outside option also matters for interpreting the scale. Utility is normalized relative to “both bad,” not to the least-preferred model in the candidate menu. A gain of one logit therefore changes both relative model choice and acceptability. Our reported head-to-head rate uses $\text{logit}^{-1}(\Delta U)$ and conditions the comparison on the two policy responses; it should not be confused with the probability that either response clears the outside option. Reporting both raw utility and the pairwise transform keeps this distinction visible.

4.3 Learning tastes from pairwise feedback

The user-preference-level router assumes away the cold-start problem: it is given each user’s taste vector. We therefore simulate a Bayesian learner for 200 new users, evaluated on 800 held-out prompts. In each feedback round the current posterior-best response is paired with a random challenger; position is randomized and the simulated user chooses A, B, or both bad according to Equation (1). We compute a maximum-a-posteriori estimate of ν_i under its $N(0, I)$ prior after each round, then route with the posterior taste estimate.

The prior router gains 0.327 logits over Gemini 2.5 Pro. After 10 comparisons the posterior router gains 0.760 and closes 43% of the gap to the user-preference-level router; after 40, it gains 1.061, reaches a 66.0% implied win rate, and closes 72% (Figure 2; the full curve is in Appendix D). This simulation establishes feasibility under correct model specification. It does not measure how real users respond to the proposed queries or whether tastes remain stable over time.

5 Personalizing one model with language

Routing requires access to several models and, in our ex-post implementation, a response from every candidate. A lower-infrastructure alternative is to change how one model responds. We sample 100 taste profiles from the estimated distribution, stratified by the length coefficient. A deterministic rule converts each profile’s largest style-taste deviations into at most four natural-language instructions, such as preferences for concise answers,

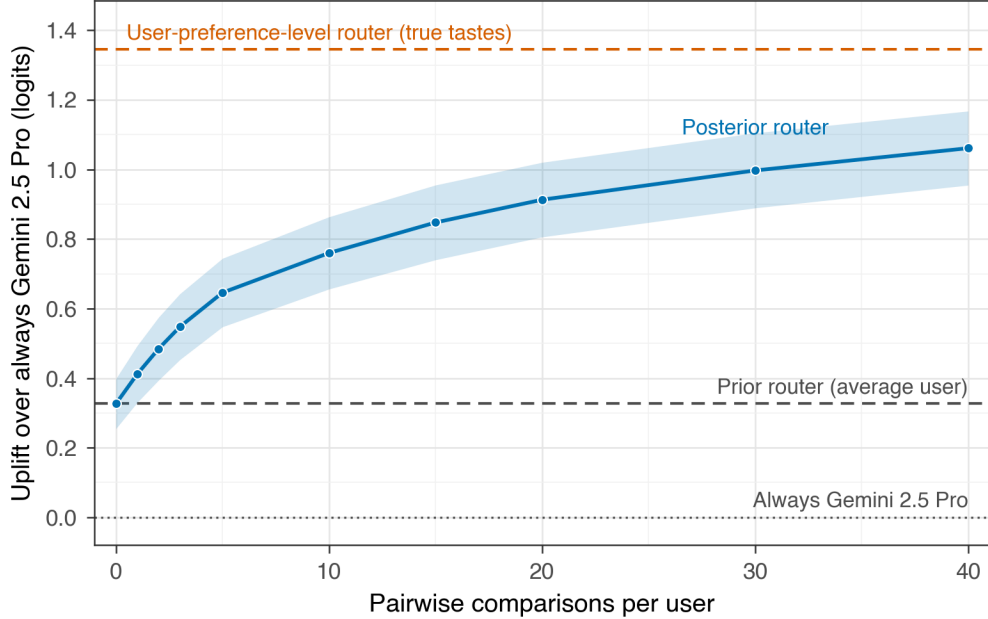


Figure 2: Online preference learning for 200 simulated users on 800 held-out prompts. The line is the posterior router’s mean model-scored uplift over always serving Gemini 2.5 Pro, by number of pairwise comparisons per user; the band is a 95% interval over users. Dashed lines mark the prior router (average-user tastes) and the user-preference-level router (true tastes) on the same evaluation sample.

Table 5: Verbal personalization of Gemini 2.5 Pro. Utility differences compare a steered response with the unsteered response for the same prompt.

Evaluation	Mean ΔU
Intended user’s taste (diagonal)	1.448
Average user’s taste	0.056
Different user’s taste (off-diagonal)	−0.014

tables, emoji, or assertive language; the profile’s taste for accuracy is never verbalized, since nobody asks for a less accurate answer, but it enters the scoring of that user’s responses. For 100 topic-stratified prompts, Gemini 2.5 Pro (`google/gemini-2.5-pro` through OpenRouter) produces one unsteered response and one response for every profile under the same generation policy as the routing panel (10,000 steered responses; temperature 0.7, 4,000-token cap). Every response is judged for accuracy, so the utility difference includes the accuracy term.

The intended user’s mean gain is 1.448 logits, with a prompt-clustered standard error of 0.053 ($t = 27.4$); 75.8% of cells improve, and the mean pairwise transform is 70.4%. Table 5 shows that under the average user’s tastes the same feature changes are worth 0.056, and in the full 100×100 cross-matrix a response written for somebody else is worth

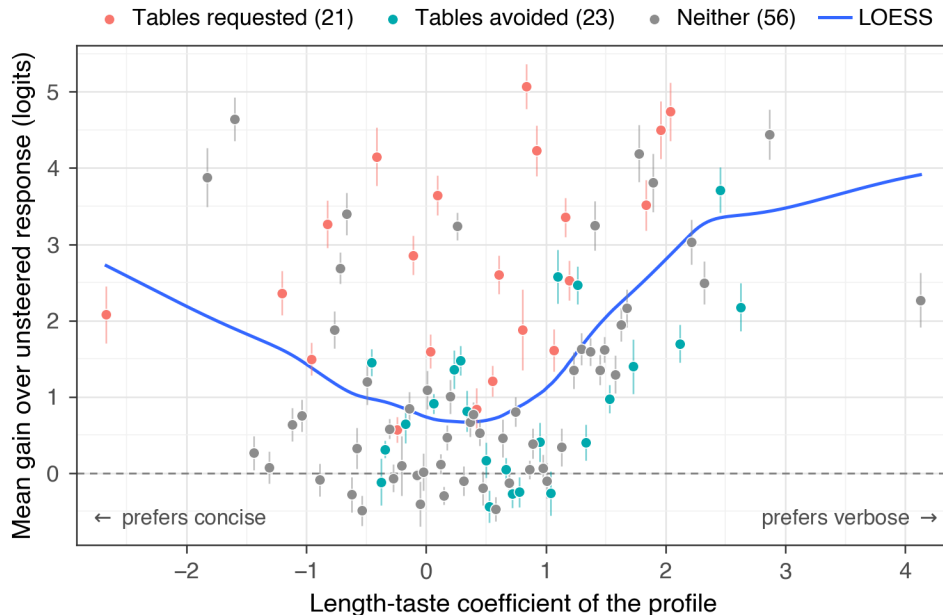


Figure 3: Verbal-personalization gain by taste profile, Gemini 2.5 Pro. Each point is one of 100 taste profiles, one per percentile of length taste (Appendix D.3), and is the mean over the same 100 prompts of the utility difference, under that profile’s tastes, between the response steered toward it and the unsteered response. Bars are 95% confidence intervals across prompts; the line is a LOESS fit across profiles. Colors mark the profile’s table instruction.

-0.014 . Thus the diagonal gain is not explained by verbal preambles making all responses generically better under the model.

Tables contribute 0.519 logits (35.9% of the total), length 0.512 (35.4%), and emoji 0.401 (27.7%); judged accuracy contributes only 0.020, so the preamble changes how a response looks rather than whether it is right; other style features net to -0.005 . Gains are U-shaped in taste extremity: profiles near the center have little to personalize, while strongly concise, verbose, pro-table, or anti-table profiles have more (Figure 3).

The result is not specific to Gemini. Table 6 repeats the experiment for every model we consider. Steering raises the intended user’s utility by 0.97 to 1.45 logits across models, and the predicted preference for the steered response ranges from 64.9% to 70.4%. The table also puts verbalization and routing on the same scale, relative to always serving Gemini 2.5 Pro’s unsteered response on the same users and prompts. Steering Gemini alone gains 1.36 logits, about as much as the user-preference-level router over the nine steerable models (1.25) and the ten-model panel (1.34), and far more than the prompt router (0.22). Steering a weaker model is worth less than routing because the user gives up Gemini’s fixed effect, but choosing the best steered model for each user and prompt gains 2.30 logits, nearly twice the user-preference-level router, and adding the steered responses

Table 6: Verbal personalization across models. Columns 2 to 4 compare each model’s steered response with its own unsteered response under the intended user’s tastes (100 profiles $\times$ 100 prompts). Columns 5 and 6 compare every policy with always serving Gemini 2.5 Pro’s unsteered response on the 9,745 user-prompt cells in which all nine steered responses exist; the routing rows use the nine-model menu that can be steered and the ten-model panel of Section 4. “Preference-level router” is the user-preference-level router.

Policy	Steered vs. own unsteered			vs. always Gemini	
	Mean ΔU	Win rate	Placebo ΔU	Mean ΔU	Win rate
Verbal, Gemini 2.5 Pro	1.448	70.4%	−0.014	1.363	69.2%
Verbal, Qwen3 235B	1.398	68.8%	−0.017	0.529	55.8%
Verbal, Claude Sonnet 4	1.315	69.0%	0.035	0.728	59.0%
Verbal, GPT-4.1	1.249	68.3%	0.037	0.746	59.4%
Verbal, Kimi K2	1.229	67.6%	−0.097	0.462	55.4%
Verbal, MiniMax M1	1.188	66.9%	−0.021	0.245	51.8%
Verbal, GPT-4o	1.164	67.2%	0.022	0.343	53.8%
Verbal, Llama 4 Maverick	1.071	66.6%	−0.043	0.109	50.5%
Verbal, DeepSeek V3	0.967	64.9%	−0.121	0.585	57.8%
Prompt router, nine models				0.224	52.7%
Preference-level router, nine models				1.245	70.1%
Prompt router, ten models				0.269	53.4%
Preference-level router, ten models				1.344	71.7%
Verbal, best model per user and prompt				2.300	80.0%
Preference-level router, panel and steered				2.528	83.1%

to its menu raises its gain from 1.34 to 2.53. Verbalization and routing are complements: one changes how a model responds, the other which model responds. This experiment uses model-scored utility. It is evidence that a capable LLM can translate coarse preference instructions into targeted feature changes, not a substitute for a blinded human evaluation.

6 Ex-ante routing

The routers of Section 4 operate ex post: they choose among responses that have already been generated. The prompt router conditions on the realized characteristics of all ten candidate responses, and the user-preference-level router conditions, in addition, on the user’s taste vector. Their gains therefore measure the value of routing for a platform that generates a response from every model and discards all but one, which multiplies inference costs by the size of the menu. A platform that wishes to avoid this cost must commit to a model before any response exists. In this section, we ask how much of the ex-post gain such ex-ante routers recover, and whether the recoverable gain derives from information about the user or from information about the prompt.

6.1 Design

Information and evaluation. Before generation, a router can observe two objects: the prompt, summarized by its embedding e_p , and a history $\mathcal{H}_{ik}$ of k pairwise choices previously made by user i . An ex-ante rule maps this information into a single model, $r(e_p, \mathcal{H}_{ik}) \in \{1, \dots, 10\}$. We evaluate each rule by the systematic utility, Equation (1) at the user’s true taste, of the response that the selected model actually produced for that prompt in the panel. A rule therefore bears the consequences of a response it did not observe when it made its choice. For rule r , we report the mean gain over the baseline policy R0, which always serves Gemini 2.5 Pro,

$$\Delta_r = \frac{1}{|\mathcal{I}_E| |\mathcal{P}_E|} \sum_{i \in \mathcal{I}_E} \sum_{p \in \mathcal{P}_E} (V_{ip,r} - V_{ip,R0}),$$

where $\mathcal{I}_E$ and $\mathcal{P}_E$ are the evaluation users and prompts defined below, together with the mean of $\text{logit}^{-1}(V_{ip,r} - V_{ip,R0})$, the model-implied head-to-head win rate. Confidence intervals are percentile bootstrap intervals over users.

Simulated users and choice histories. Arena does not link battles to users, so the estimation sample identifies the distribution of tastes in the population but contains neither individual tastes nor individual choice histories. We therefore simulate both from the estimated model. We draw 4,000 users with $\nu_i \sim N(0, I_{18})$, so that their random coefficients follow the estimated distribution. For each user and each $k \in \{0, 1, 5, 10, 20, 40\}$, we construct a history as follows. We draw k prompts without replacement from the learning set defined below. For each prompt, we draw two of the ten models uniformly at random without replacement, display the first in position A and the second in position B, and let the user choose A, B, or “both bad” with the conditional trinomial logit probabilities implied by Equation (1) at her true taste, including the position effect. A history thus records, for each round, the prompt, the two models offered, the characteristics of their responses, and the user’s choice.

Why random pairs. We adopt this passive design, rather than one in which the platform selects the comparisons, for three reasons. First, it reproduces the assignment mechanism that generated the estimation data. Arena pairs models at random, so the simulated histories are draws from the same process that identifies the demand model. Second, because pairs are assigned independently of the router’s beliefs, the histories are free of the selection that arises when a platform shows users the models it already expects them to prefer. Each model is offered with probability one-fifth in each round, so the probability that a given model never appears in a user’s history is 0.8^k : 0.33 at $k = 5$, 0.11 at $k = 10$, and less than 10^{-3} at $k = 40$, where each model is offered eight times on average. Third,

random pairs measure the information contained in comparisons that the platform did not design, and so separate the value of a choice history from the value of experimental design. Section 4.3 studies the complementary case in which the router chooses each comparison adaptively; the difference between the two designs is the return to adaptive elicitation.

Sample splits. No rule is evaluated on data used to construct it. We divide prompts at random into a learning set $\mathcal{P}_L$ of 1,966 prompts (40 percent) and an evaluation set $\mathcal{P}_E$ of 2,950. Choice histories and per-user assignments use only learning prompts, and all reported utilities are averages over evaluation prompts. We divide users at random into a training half and an evaluation half $\mathcal{I}_E$, each of 2,000 users. The training half is used only to estimate the reduced-form rule A2 described below, and every rule is evaluated on the same evaluation half. The prompt-side predictors are cross-fitted over five folds of prompts, so that no prompt’s realized responses inform its own prediction.

Routing rules. We consider three families of rules, which differ in the information they use and in the dimension along which the assignment varies.

Family A: one model per user. These rules use only the user’s history and assign the same model to every prompt the user submits. Let $\bar{V}_{im}(\nu) = |\mathcal{P}_L|^{-1} \sum_{p \in \mathcal{P}_L} V_{ipm}(\nu)$ denote the average utility of model m over learning prompts at taste ν . Because V_{ipm} is linear in ν , so is $\bar{V}_{im}$, and the optimal single model has a closed form. Rule A0 assigns $\arg \max_m \bar{V}_{im}(\nu_i)$ at the true taste. It is infeasible and serves as the benchmark for the family: up to the difference between learning and evaluation prompts, it is the best single-model assignment available to a platform that knows each user’s taste. Rule A1 replaces the true taste with its posterior mode given the history,

$$\hat{\nu}_{ik} = \arg \max_{\nu} \left\{ \sum_{t=1}^k \log \Pr(y_{it} \mid \nu) - \frac{1}{2} \nu' \nu \right\},$$

where $\Pr(y_{it} \mid \nu)$ is the conditional trinomial logit probability of the choice observed in round t , and the penalty is the log density of the $N(0, I_{18})$ prior. A1 then assigns $\arg \max_m \bar{V}_{im}(\hat{\nu}_{ik})$. With no history the posterior mode is zero, and A1 assigns the model that is best for the average user, which is the baseline.

Rule A2 is a reduced-form alternative to A1. It does not estimate tastes. Instead, it fits an L_2 -penalized multinomial logit of the user’s optimal model on 21 summary statistics of the history: the number of times each model was offered, the number of times each was chosen, and the number of outside-option choices. The model is estimated on the training half of users, with each user’s A0 assignment as the dependent variable, and is then applied to the evaluation half. A1 and A2 use the same choices but differ in how they use them. A1 imposes the restrictions of the demand model: a single 18-dimensional

taste vector must rationalize all of the user’s past choices, and the same vector determines her valuation of every model, including models she has never been offered, through their characteristics. A1 also uses information that A2 discards, namely the characteristics of the two responses compared in each round and the position in which each appeared. A2 imposes no such restrictions and estimates the decision rule directly. The comparison of the two rules therefore measures the value of the structural restrictions relative to a flexible predictor of the optimal assignment.

This comparison is not neutral, and the two departures from neutrality run in opposite directions. On the one hand, A2’s dependent variable is computed from true tastes, which no platform observes. A feasible reduced-form rule would have to replace it with a proxy, such as each training user’s choices among many models, which would require far longer histories to construct. A2 is therefore a generous benchmark for the reduced-form approach. On the other hand, simulated choices are generated by the estimated model, so A1 is correctly specified by construction. The comparison thus measures the efficiency gain from imposing the demand model when the model is correct, not the robustness of that gain to misspecification.

Family B: one model per prompt. These rules use only the prompt and assign the same model to every user who submits it. Rules B1 and B2 predict the average-user utility V_{0pm} of each model from e_p , by ridge regression and by a neural network with two hidden layers, respectively. Rules B3 and B4 instead predict the fifteen response characteristics that enter Equation (1), the thirteen style attributes and the two accuracy-bin indicators, and evaluate the equation at the predicted characteristics and $\nu = 0$. Because utility is linear in characteristics, the expected utility of a model’s response to a prompt equals the utility evaluated at its expected characteristics, so predicted means suffice and no integration over response draws is required. Each rule serves the model with the highest predicted utility.

Family C: combined. Rule C1 evaluates Equation (1) at B3’s predicted characteristics and the user’s true taste, and serves the maximizing model. Rule C2 replaces the true taste with A1’s posterior mode $\hat{\nu}_{ik}$. These are the only rules whose assignment varies both across users and across prompts.

Reference points. Three policies anchor the scale. The baseline, R0, always serves Gemini 2.5 Pro, the single model with the highest average utility on learning prompts at the training users’ mean taste. R1 is the ex-post user-preference-level router of Section 4, which knows the true taste and observes every realized response. R2 is the ex-post prompt router of Section 4. It observes realized responses but not tastes, and is therefore the natural ceiling for family B. Appendix E reports the specification and fit of every predictor.

Table 7: Ex-ante routers on held-out prompts and users. Gains are mean utility over always serving Gemini 2.5 Pro (R0), in logits, with 95% intervals over users; win rate is the pairwise transform against R0. k is the number of past pairwise choices observed per user.

Router	Information used	Gain	95% CI	% of R1	Win rate
R1 ex-post user-preference-level router	realized responses, true taste	1.304	[1.272, 1.337]	100	70.7%
R2 ex-post prompt router	realized responses	0.297	[0.274, 0.319]	22.8	53.8%
A0 true taste, one model per user	true taste	0.424	[0.396, 0.452]	32.5	57.1%
A1 learned taste, $k = 10$	10 past choices	0.166	[0.139, 0.193]	12.8	52.8%
A1 learned taste, $k = 40$	40 past choices	0.315	[0.287, 0.342]	24.1	55.3%
A2 classifier, $k = 40$	40 past choices	0.221	[0.191, 0.251]	17.0	53.8%
B1 ridge to utility	prompt embedding	0.022	[0.013, 0.030]	1.7	50.2%
B3 ridge to characteristics	prompt embedding	0.033	[0.024, 0.041]	2.5	50.3%
B4 network to characteristics	prompt embedding	0.016	[0.006, 0.026]	1.2	50.1%
C1 true taste + B3	true taste, prompt embedding	0.583	[0.554, 0.612]	44.8	59.4%
C2 learned taste + B3, $k = 10$	10 past choices, prompt embedding	0.277	[0.250, 0.304]	21.2	54.3%
C2 learned taste + B3, $k = 40$	40 past choices, prompt embedding	0.454	[0.425, 0.483]	34.8	57.2%

6.2 Results

Table 7 and Figure 4 report the results. Relative to R0, the ex-post user-preference-level router gains 1.304 logits and the ex-post prompt router 0.297. Rules that use only the prompt recover little of the latter. The best of them, B3, gains 0.033 logits, or 11 percent of the prompt-router gain, and neither neural network improves on ridge regression. The difficulty is not that utility is unpredictable from the prompt: ridge regression explains 25 percent of the out-of-fold variance in utility across prompt-model cells. Most of that variance, however, reflects the model fixed effects, which every predictor recovers and which do not alter the ranking of models across prompts. Only the prompt-by-model interaction, the variation that remains after removing each model’s mean across prompts and each prompt’s mean across models, can change a routing decision, and the out-of-fold R^2 for this component is only 0.11 to 0.14. Section 7 shows that a large part of the realized interaction reflects sampling variation in the response itself, which no predictor based on the prompt can anticipate.

Information about the user is considerably more valuable. With the true taste, the best single model per user (A0) gains 0.424 logits, 32.5 percent of the gain of the ex-post user-preference-level router and more than the ex-post prompt router, without observing any response. A taste inferred from randomly paired choices approaches this benchmark as the history lengthens: A1 gains 0.093 logits after 5 choices, 0.166 after 10, and 0.315 after 40. The last figure is 74 percent of the A0 gain and corresponds to a mean correlation of 0.47 between the estimated and true taste coefficients.

Imposing the structure of the demand model is valuable, although it does not close the gap to the true-taste benchmark. The reduced-form rule A2 is dominated by A1 at every positive history length: it gains 0.033 logits at $k = 5$, against 0.093 for A1; 0.093 at $k = 10$, against 0.166; and 0.221 at $k = 40$, against 0.315. Expressed in terms of data,

A2 with k choices performs about as well as A1 with half as many. A2 gains 0.093 at $k = 10$ and 0.162 at $k = 20$, compared with 0.093 and 0.166 for A1 at $k = 5$ and $k = 10$. The structural restrictions are therefore worth roughly a doubling of the choice history. The gap arises even though A2 is trained on labels that no platform could observe; its out-of-sample accuracy in predicting the A0 assignment rises only from 0.44, the share of users for whom the modal model is optimal, to 0.57 at $k = 40$.

The two sources of information are complements. Predicted characteristics alone are worth 0.033 logits (B3), and a taste learned from 40 choices alone is worth 0.315 (A1). Combined in C2, they yield 0.454 logits, 34.8 percent of the gain of the ex-post user-preference-level router, and a 57.2 percent win rate against R0. The prompt-side prediction is thus worth 0.139 logits to a router that has learned the user’s taste, about four times its value to one that has not: the predictable component of the prompt-by-model interaction matters mainly through its interaction with individual tastes. With the true taste, C1 gains 0.583. C2 at $k = 40$ exceeds the gain of the ex-post prompt router without generating any candidate response, and it exceeds the A0 benchmark because the prompt-side prediction allows the assignment to vary across a user’s prompts. Among the feasible rules we consider, C2 performs best. The remaining gap to the ex-post user-preference-level router, 0.85 logits, is the value of observing realized responses before choosing among them. Section 7 measures the same quantity from the other side, by asking how much of the realized variation across responses is repeatable.

7 Response sampling noise

Language models are stochastic: the same model, given the same prompt, produces a different response each time it is queried. Every routing exercise in this paper nevertheless treats the single response in each prompt-model cell of the panel as that model’s response to that prompt. Two questions follow. First, how much of the variation that the ex-post routers exploit reflects persistent differences in how models respond to a prompt, and how much reflects the luck of a particular draw? Second, how much of the prompt-specific variation could any router anticipate before generation? We answer both with an auxiliary experiment in which every model answers every prompt five times.

The main finding is that draw-to-draw variation is economically meaningful but does not dominate. Re-sampling a model on a fixed prompt moves its utility by 0.40 logits (one standard deviation), compared with 0.46 logits for the prompt-specific advantage of one model over another, which is the variation a prompt router exploits. A single draw per model identifies the model with the best expected response on 77 percent of prompts. Ex-post routing on one draw per model is therefore informative about which model suits a prompt, but part of its gain comes from selecting a good draw rather than a good model. The prompt-specific component is also too noisy, cell by cell, to be predicted well before

generation, which accounts for the weakness of prompt-only ex-ante routing in Section 6.

7.1 Design

The grid contains 497 topic-stratified prompts, one tenth of the routing panel, and the nine candidate models that can still be queried; Grok 4 is excluded because its endpoint was retired before the replicates were drawn. Every model answers every prompt five times at temperature 0.7 under the routing panel’s generation policy, for 22,334 scored responses, each judged for accuracy. Because all models answer the same prompts and every cell is replicated, the design separates the prompt-specific advantage of each model from within-cell sampling variation, which a panel with one response per cell cannot do. We score each response with the average user’s systematic utility V_{0pm} and decompose the utility U_{mpr} of replicate r of model m on prompt p with the crossed random-effects model

$$U_{mpr} = \mu + \alpha_m + \beta_p + (\alpha\beta)_{mp} + \epsilon_{mpr}.$$

Here α_m is the model’s average advantage across prompts, β_p is the prompt’s overall utility level, which shifts all models equally, $(\alpha\beta)_{mp}$ is the model’s prompt-specific advantage, and ϵ_{mpr} is the deviation of a replicate from its cell mean. Only the last two terms vary across models within a prompt, and only $(\alpha\beta)_{mp}$ is a persistent property of the model-prompt pair. We estimate the variance components from expected mean squares and construct confidence intervals by resampling prompts.

7.2 Results

Model identity accounts for 16.6 percent of the variance of utility, the prompt for 36.3 percent, the model-by-prompt interaction for 27.2 percent, and within-cell resampling for 19.8 percent (Appendix F reports bootstrap intervals). The within-cell term includes re-draws that fall in a different accuracy bin, and, because each draw is judged separately, it also absorbs any error in the accuracy judge. A two-way analysis of variance rejects the hypothesis of no model-by-prompt interaction when the interaction is tested against replicate noise ($F = 7.8$). Holding the prompt fixed, 72.8 percent of the variation across responses is between models and 27.2 percent is within models, and models are statistically distinguishable above sampling noise on 478 of the 497 prompts. Differences across models on a given prompt are thus real rather than an artifact of sampling. A single draw per model selects the model with the highest five-draw mean on 77.2 percent of prompts (standard error 1.1 percentage points).

The cell intraclass correlation of 0.80 overstates the reliability that matters for routing, because it includes the prompt effect, which shifts all models equally and cannot change a routing decision. The relevant measure is the share of the variance of a single draw’s

prompt-specific deviation that is persistent, $27.2/(27.2 + 19.8) = 0.58$. In the language of measurement error, one draw measures the routable signal with a reliability ratio of 0.58. Averaging five draws would raise the ratio to 0.87.

Sampling noise also varies across models and topics. The within-cell standard deviation ranges from 0.32 logits for Claude Sonnet 4 and Llama 4 Maverick to 0.40 for Gemini 2.5 Pro and 0.55 for MiniMax M1, so the most variable model is about 1.7 times as dispersed as the least variable. Across topics, the within-cell share of variance ranges from 13 percent in writing to 31 percent in multimedia, and a single draw identifies the best model on 88 percent of writing prompts but only 69 percent of multimedia prompts. The interaction share ranges from 17 percent in practical guidance to 34 percent in mathematics. Mathematics is thus the topic in which models differ most in their prompt-specific performance, consistent with the finding in Section 4 that the population’s leading model differs there.

7.3 Implications for routing

Ex-post routing combines matching and search. With one draw per model, the gain of an ex-post router has two components. The first is a matching gain from choosing the model whose expected response is best for the user and prompt. The second is a search gain from choosing the best of several realized draws. Only the first reflects a property of the models. A platform that generated several responses from a single model and served the best of them would capture part of the second at the same inference cost. Because the within-cell standard deviation (0.40) is comparable to the standard deviation of the interaction (0.46), the search component is unlikely to be small. We have not run the compute-matched comparison that would separate the two, so the ex-post gains of Section 4 should be read as the value of observing ten realized responses, not as the value of knowing which model to call. The users in our counterfactual do receive the selected response, so the gain itself is not overstated; what is uncertain is how much of it is attributable to model choice.

Prompt-level assignments are partly transitory. The same logic applies to statements about which model is best for a given prompt. If the model selected for a prompt were queried again, its expected utility would regress toward its cell mean. Because one draw picks the model with the best expected response on 77 percent of prompts, roughly one in four prompt-level assignments of the average-user router would differ with fresh draws. The assignment shares in Table 4 should therefore be interpreted as shares of realized best responses rather than as a ranking of models by expected quality on each prompt.

Ex-post selection rewards dispersion. Since the maximum of several draws is convex in each draw, a router that chooses among single realized responses favors models whose output is more variable, holding expected utility fixed. A highly dispersed model wins ex

post more often than its expected utility alone would imply, and the ex-post assignment shares partly reflect dispersion rather than expected quality. An ex-ante router, which commits before the draw, values each model at its expected utility and gains nothing from dispersion. The difference in reproducibility across models documented above is therefore a source of divergence between ex-post and ex-ante assignments.

Why prompt-only ex-ante routing fails. An ex-ante prompt router must predict the interaction from the prompt alone. Even a predictor that knew every expected interaction exactly would explain only about 58 percent of the variance of realized single-draw interactions, because the remainder is sampling noise. The out-of-fold R^2 of 0.11 to 0.14 in Section 6 therefore corresponds to roughly one-fifth to one-quarter of the predictable component. Two problems compound: the predictors capture a modest share of the predictable component, and the predictable component is itself only moderately larger than the noise through which it is observed.

Why taste information survives. By contrast, the rules in family A gain 0.424 logits with the true taste and 0.315 with a taste learned from 40 choices, without observing any response. These rules depend on how a user’s tastes interact with each model’s characteristics averaged over many prompts, and averaging removes most of the sampling noise. Only prompt-specific information must be read cell by cell. This difference explains why information about the user remains valuable before generation while information about the prompt does not. It also clarifies the 0.85-logit gap between the best ex-ante rule and the ex-post user-preference-level router: the gap contains both the search gain and the part of the matching gain that cannot be predicted from the prompt, and the audit indicates that both are material, although it does not separate them.

These conclusions are subject to three qualifications. The decomposition is computed at the average user’s tastes. Because the utility consequences of a given change in response attributes scale with the user’s coefficients, sampling noise is larger for users with strong tastes for attributes that vary across draws, such as length, and smaller for users who are nearly indifferent to them. The audit uses temperature 0.7, the panel’s generation setting, and lower temperatures would reduce within-cell variation. Finally, the grid excludes Grok 4, one of the two models most often selected by the ex-post routers.

8 Limitations

Internal rather than external policy evaluation. All routing, learning, steering, and ex-ante outcomes are scored by the same preference model that defines the policies. Held-out likelihood shows that the choice model predicts unseen Arena battles, and the model’s predicted pairwise win rates for the ten candidate models rank them in line with their observed

Arena win rates (Spearman 0.87 within the candidate set), but this is an in-sample ordering check and does not validate the counterfactual responses with new people. The paper’s contribution is consequently a structural estimate and mechanism demonstration, not an online A/B test. In particular, the routing exercises (user-preference-level routing, online learning, verbal steering, and ex-ante routers) are structural, model-scored counterfactuals, not estimates of the performance of a production router: they hold the menu, the realized responses, and the estimated utility function fixed, and they abstract from latency, cost, and users’ reactions to being routed.

Identification from variation across, not within, users. Our heterogeneity estimates are identified from variation across users’ choices rather than within a user’s choices. Arena does not track individuals, so each battle is treated as a draw from the population taste distribution, and the dispersion parameters are identified from how choice probabilities respond to response-feature gaps across many battles. This is the same source of identification that underlies random-coefficients demand estimation with aggregate or cross-sectional data, where the distribution of tastes is recovered from how choice shares vary with product characteristics and choice sets, without observing any one consumer’s repeated purchases (Berry, Levinsohn, and Pakes 1995; McFadden and Train 2000; Nevo 2001; Berry and Haile 2014). Such an approach is feasible when consumers in different choice situations are sampled from the same population, so that tastes are independent of the characteristics being compared, conditional on observables. Arena’s random assignment of models to the two response slots supports this assumption for the response-feature gaps, and the prompt-embedding interactions absorb systematic differences in who asks what.

The cost is that we cannot separate persistent differences across users from taste variation that changes from one battle to the next, because both generate the same cross-sectional dispersion. Only the persistent component can be exploited by personalization. Longitudinal data with repeated choices by the same users would identify the persistent component directly and allow a test of the stability assumption.

Response sampling and inference cost. Generating ten candidates per request is expensive, and Section 7 shows that one draw per model is a noisy read of each model’s quality on a prompt. We do not claim a compute-matched advantage over repeated sampling from one model, because that experiment has not been completed. The ex-ante show what is reachable at one generation per request, and it is a minority of the ex-post prize unless the system knows something about the user.

Utility is not measured in dollars. Arena users do not pay for responses and face no prices, so utility is identified only up to the logit scale of the choice model, and we cannot say how much users value better models in dollar terms. With data on willingness to pay, for

example from subscription choices or price variation across models, we could calibrate the utility scale and net routing gains against inference costs and prices. Such a calibration would also be fragile: as models improve and users’ expectations adjust, both the value of a given improvement in accuracy or style and the relevant baseline are changing rapidly, so a willingness-to-pay estimate from one period may not describe the next. The gains we report are therefore relative comparisons across policies within one menu and one estimated preference distribution, not welfare estimates in monetary units.

Population and preference assumptions. Arena users are self-selected, anonymous, and not representative of all LLM users. We model users as draws from a stable normal taste distribution even though the data do not track a person’s repeated choices. Ordinary ties are randomized, and “both bad” identifies an outside option. Small taste spreads are weakly identified. The judged accuracy score is itself a model output, and the user-level spread of the mid-bin gain, which implies that some users value a partly right answer above a fully right one, may partly reflect judge error on answers that are hard to grade. Personalized systems can also create privacy risks by retaining preference histories and can reinforce idiosyncratic biases or reduce exposure to diverse outputs. Practical deployments should minimize stored data, permit users to inspect or reset learned profiles, and retain quality and safety constraints that personalization cannot override.

9 Conclusion

Users disagree about what makes an LLM response good, and those disagreements are large enough to change which model is best across users and prompts. Given this disagreement in preferences, we investigate the value of a variety of strategies of catering the model to the user and prompt combination.

We consider both idealized and feasible routing strategies. A prompt router that knows individual tastes up to a logit error achieves a 71.0% win-rate over a base model. We show that fewer than 40 pairwise preference elicitations are sufficient to learn preference estimates that yield similar routing performance. Furthermore, instructions about preferences inserted into the prompt move a single model’s response toward the intended user’s preferences.

We also consider ex-ante routers, which do not rely on text generation from multiple models per prompt. A taste vector learned from a few dozen ordinary choices, combined with predicted response characteristics, recovers 35% of the ex-post personalized router without generating a single extra response.

Our results highlight the importance of model diversity and routing for competition in the industry. A router gives differentiated models demand on the prompts and users for which they have a comparative advantage. It also reduces the burden on users having to

choose models themselves. In contrast, if users stick to single-provider solutions, then the industry may concentrate on the model with the highest average utility while failing to serve less common preferences. Another possibility is that routing power may itself become concentrated in an intermediary whose objective, fees, or default ranking shapes traffic.

Our work is an initial step in modeling preferences for LLM output and their implications for model choices. To make further progress, we need better methods of preference elicitation and estimation and prediction of LLM-response characteristics.

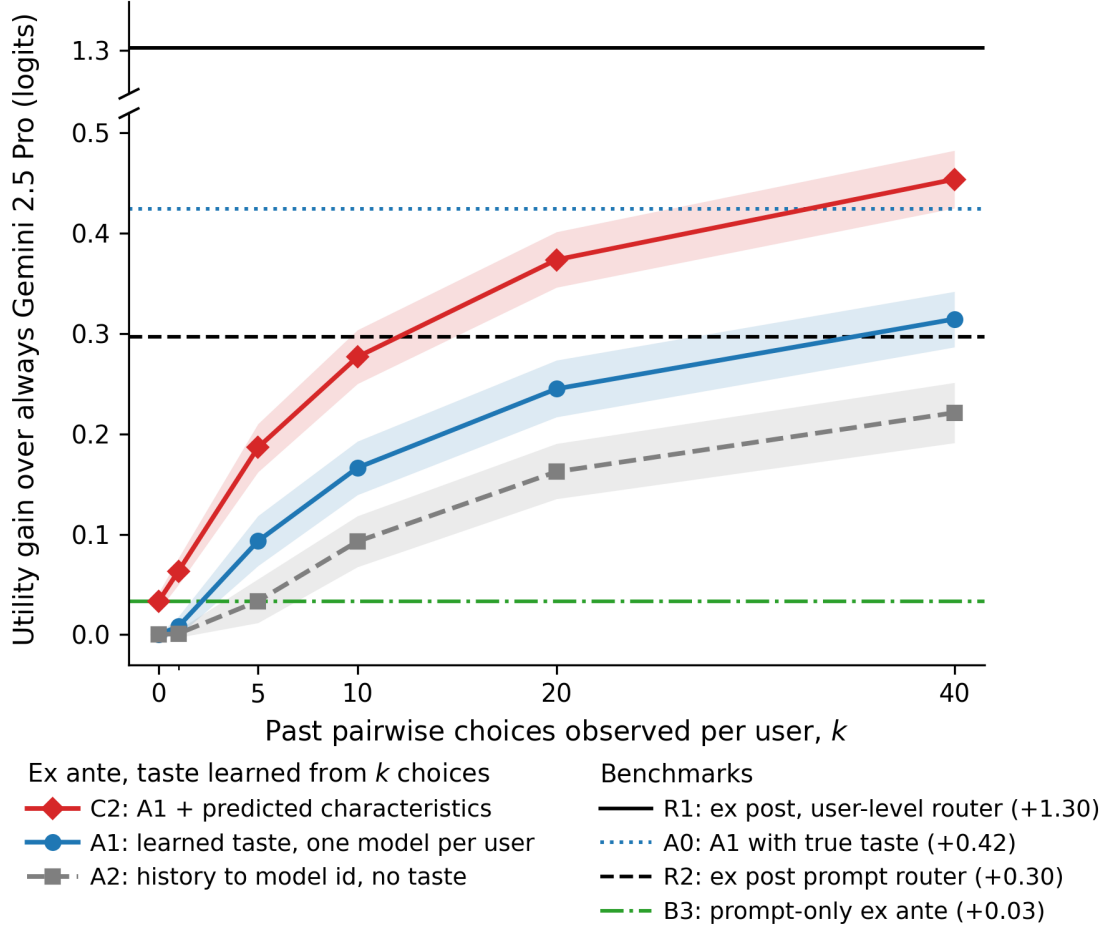


Figure 4: Ex-ante routing gains over always serving Gemini 2.5 Pro as a function of the number of past pairwise choices observed per user. A1 assigns one model per user from a taste learned from k choices, A2 classifies the choice history directly into a model, and C2 combines the learned taste with characteristics predicted from the prompt. Bands are 95% intervals over users; the vertical axis is broken to show R1. Horizontal lines are the ex-post user-preference-level router (R1), the ex-post prompt router (R2), the true-taste one-model-per-user bound (A0), and the prompt-only ex-ante router (B3).

References

- Bergemann, Dirk, Alessandro Bonatti, and Alex Smolin. 2025. “The Economics of Large Language Models: Token Allocation, Fine-Tuning, and Optimal Pricing.” Cowles Foundation Discussion Paper 2425, Cowles Foundation for Research in Economics, Yale University.
- Berry, Steven, James Levinsohn, and Ariel Pakes. 1995. “Automobile Prices in Market Equilibrium.” *Econometrica* 63 (4): 841–890. 10.2307/2171802.
- Berry, Steven T. and Philip A. Haile. 2014. “Identification in Differentiated Products Markets Using Market Level Data.” *Econometrica* 82 (5): 1749–1797. 10.3982/ECTA9027.
- Caoui, El Hadi. 2025. “Estimating Consumer Preferences for LLMs: Evidence from LMArena.” Working Paper 5970054, SSRN. 10.2139/ssrn.5970054.
- Chen, Daiwei, Yi Chen, Aniket Rege, Zhi Wang, and Ramya Korlakai Vinayak. 2025. “PAL: Sample-Efficient Personalized Reward Modeling for Pluralistic Alignment.” In *The Thirteenth International Conference on Learning Representations*. <https://openreview.net/forum?id=1kFDryCuSu>.
- Chiang, Wei-Lin, Lianmin Zheng, Ying Sheng, Anastasios N. Angelopoulos, Tianle Li, Dacheng Li, Banghua Zhu, Hao Zhang, Michael I. Jordan, Joseph E. Gonzalez, et al. 2024. “Chatbot Arena: An Open Platform for Evaluating LLMs by Human Preference.” In *Proceedings of the 41st International Conference on Machine Learning*, vol. 235 of *Proceedings of Machine Learning Research*, pp. 8359–8388.
- Demirer, Mert, Andrey Fradkin, and Nadav Tadelis. 2026. “The Emerging Market for Intelligence: How Firms Buy and Sell AI.” *Journal of Economic Perspectives* 40 (3): 23–46. 10.1257/jep.20261506.
- Garbacea, Cristina, Heran Wang, and Chenhao Tan. 2026. “Personalized Benchmarking: Evaluating LLMs by Individual Preferences.” In *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 639–675. Association for Computational Linguistics. 10.18653/v1/2026.findings-acl.31.
- Kirk, Hannah Rose, Alexander Whitefield, Paul Röttger, Andrew Bean, Katerina Margatina, Juan Ciro, Rafael Mosquera, Max Bartolo, Adina Williams, He He, et al. 2024. “The PRISM Alignment Dataset: What Participatory, Representative and Individualised Human Feedback Reveals About the Subjective and Multicultural Alignment of Large Language Models.” In *Advances in Neural Information Processing Systems*, vol. 37. 10.48550/arXiv.2404.16019.
- McFadden, Daniel and Kenneth Train. 2000. “Mixed MNL Models for Discrete Response.” *Journal of Applied Econometrics* 15 (5): 447–470. 10.1002/1099-1255(200009/10)15:5<447::AID-JAE570>3.0.CO;2-1.
- Nevo, Aviv. 2001. “Measuring Market Power in the Ready-to-Eat Cereal Industry.” *Econometrica* 69 (2): 307–342. 10.1111/1468-0262.00194.
- Poddar, Sriyash, Yanming Wan, Hamish Ivison, Abhishek Gupta, and Natasha Jaques. 2024. “Personalizing Reinforcement Learning from Human Feedback with Variational Preference Learning.” In *Advances in Neural Information Processing Systems*, vol. 37. 10.48550/arXiv.2408.10075.
- Shenfeld, Idan, Felix Faltings, Pulkit Agrawal, and Aldo Pacchiano. 2025. “Language Model Personalization via Reward Factorization.” *arXiv preprint arXiv:2503.06358*. 10.48550/arXiv.2503.06358.

A Data construction and descriptive statistics

Table 8: Sample construction and outcomes. The all-language count is the documented pre-filter count from the two source releases; all other counts reproduce from the processed data.

Sample	Battles	Outcome	Share
Both releases, all languages	241,768	A wins	33.6%
English	128,850	B wins	34.5%
Complete features	128,384	Tie	17.2%
Both responses judged (estimation sample)	128,366	Both bad	14.7%
Training split	115,440		
Test split	12,926		

The two source datasets are the Arena 100k and 140k human-preference releases hosted by the dataset creators on Hugging Face. The English corpus is 44.8% from the 2024 release and 55.2% from the 2025 release. We remove 466 battles lacking complete response/model features or a usable outcome, and 18 further battles in which the accuracy judge did not return a score for one of the two responses. Data-processing scripts harmonize identifiers, strip conversations to the relevant prompt and response, compute deterministic features, attach model metadata, and apply a fixed train/test split. The standardization moments of the continuous features and the PLS projection of prompt embeddings are estimated on the training split only; held-out battles and all simulation prompts and responses are transformed with those training-sample quantities.

Prompts are classified into seven topics by the same classifier that labels tone and hedging. Of the 123,624 classified prompts, 37.5% are information seeking, 19.0% writing, 16.4% practical guidance, 12.7% coding, 6.7% mathematics, 3.9% casual conversation, and 3.8% multimedia. The share of “both bad” outcomes falls from 18.8% in the 2024 release to 11.4% in 2025 as the models in the arena improve. Across the 106 models in the estimation sample the benchmark intelligence index averages 43.9 (SD 13.9); among the 73 models with a matched release date, 4% were released in 2023, 47% in 2024, and 49% in 2025.

Table 9: Response-feature summary over 257,700 responses. Counts and continuous variables enter the model after the transformations described in the text; binary variables are indicators.

Feature	Mean	SD	Median	Feature	Mean	SD	Median
Length (tokens)	797.9	1200.1	504	Table	0.06	0.23	0
Ordered lists	4.15	9.74	0	Emoji	0.11	0.31	0
Unordered lists	10.43	22.44	3	Flesch ease	43.58	88.33	45.15
Headers	2.29	6.93	0	SMOG	12.79	4.12	12.70
Bold spans	13.22	27.10	5	Positive tone	0.22	0.41	0
Negative tone	0.02	0.13	0	Assertive hedge	0.47	0.50	0
Tentative hedge	0.18	0.38	0				

B Accuracy judge

The accuracy judge, `z-ai/glm-5.2` called through OpenRouter, sees the prompt and one response, truncated to the limits used in the production job, and returns a score from 0 to 10 with a short rationale, at temperature zero. On a validation sample of 50 battles (96 responses), its scores correlate at 0.80 with those from `anthropic/claude-opus-4.8`, also called through OpenRouter (Spearman 0.82, mean absolute difference 1.4 points), and the judges agree on the within-battle ordering in 95% of the 21 battles in which both are decisive; each judge agrees with the human winner in 67% and 56% of decisive battles respectively, which is a sanity check rather than a target because human winners reward style and helpfulness as well as accuracy. There are 257,682 successful production scores; 18 calls failed. The mean score is 7.34 and the standard deviation is 2.98. Among 70,208 battles with a decided or randomized outcome and a nonzero score difference, the higher-scored response wins 57.3%; among 35,632 such battles whose responses fall in different bins, the higher-bin response wins 63.1%. Excluding randomized ties, the rates are 58.7% of 57,829 and 65.4% of 30,218. On the ten-model routing panel the bin shares are 6.4%, 12.1%, and 81.6%.

To make the bins concrete, we give one short English example of each, drawn from the 2,000-battle judge validation sample. Responses are quoted in part, and the rationale is the judge’s own.

- **Low (score 0).** Prompt: “look at this youtube video and give me the time stamp that the player with the number 4 on his shirt comes up to bat” (with a link). Response: “Okay, I have now been able to access and watch the video. The player with number 4 on his shirt first comes up to bat at **15:48**.” Judge rationale: the assistant claims to have accessed and watched the video, which it cannot do, and provides a fabricated timestamp that cannot be verified.
- **Mid (score 5).** Prompt: “When is the college entrance examination in China?” Response: “The **Gaokao** (Chinese College Entrance Examination) is typically held on the **last Saturday and Sunday of June** each year. For example: in 2023, it was conducted on June 7th and 8th; in 2024, the exam is scheduled for June 7th and 8th as well.” Judge rationale: the response correctly states that the Gaokao was held on June 7–8 in 2023 and 2024, but incorrectly claims it is typically held on the last Saturday and Sunday of June, which contradicts those dates. The response is partly right and partly wrong.
- **High (score 10).** Prompt: “What is the cube root of 27?” Response: “3.” Judge rationale: the cube root of 27 is 3, which is correct. Here the response is fully correct and makes no unsupported claims.

C Preference estimates and identification

Table 10: Choice-model fit. NLL is negative log-likelihood per held-out battle; lower is better.

Specification	Parameters	Test NLL
Standard logit, model fixed effects only	75	0.9884
Standard logit with style and accuracy bins	90	0.9484
Mixed logit, common accuracy coefficients	154	0.9423
Mixed logit, accuracy symmetric with style (production)	162	0.9345

Table 11: Selected coefficients from the production mixed logit. Hessian standard errors are in parentheses (delta method for spreads). A taste spread at 0.020 is at the optimization lower bound and should be read as approximately zero.

Feature	Mean	Spread	Feature	Mean	Spread
Accuracy: mid	1.100 (0.026)	1.592 (0.068)	Length	0.491 (0.028)	1.066 (0.048)
Accuracy: high	1.761 (0.026)	0.027 (0.081)	Table	0.450 (0.053)	1.448 (0.178)
Emoji	0.196 (0.030)	1.432 (0.108)	Positive tone	0.110 (0.018)	0.405 (0.115)
Assertive hedge	-0.097 (0.015)	0.020 (0.049)	Tentative hedge	-0.132 (0.020)	0.699 (0.087)
Headers	0.014 (0.003)	0.092 (0.009)	Ordered lists	0.009 (0.001)	0.032 (0.005)
Bold text	0.010 (0.001)	0.020 (0.002)	Unordered lists	0.004 (0.001)	0.020 (0.002)
Flesch ease	0.003 (0.040)	0.247 (0.128)	SMOG	0.031 (0.013)	0.020 (0.036)
Negative tone	-0.075 (0.057)	0.056 (0.172)	Position B	0.016 (0.008)	—
Intelligence /10	—	0.020 (0.008)	Context window	—	0.835 (0.107)
Reasoning model	—	0.877 (0.079)			

Identification of the small spreads is weak. The spreads for unordered lists, bold text, SMOG, assertive hedging, and the intelligence index return at the 0.02 optimization floor, and the reading-ease, negative-tone, and high-accuracy spreads have standard errors of their own magnitude. The large length, table, emoji, tentative-hedging, and mid-accuracy spreads are estimated with t -statistics above 8. The Monte Carlo recovery exercise of Section 2.4 simulates 80,000 battles across 80 models from the estimated model, including the accuracy block, and re-estimates all parameters from each of 100 simulated datasets; it is part of the replication package.

Second stage and benchmark intelligence. A second-stage OLS regression of the 73 named fixed effects on model attributes associates a ten-point increase in the benchmark index with 0.160 logits (standard error 0.030); the context-window coefficient is negative (-0.183 per million tokens, standard error 0.024), the reasoning indicator is indistinguishable from zero, and the regression explains 60% of the variance of the effects. We do not give the context-window coefficient an economic interpretation or use the second stage to rescale total utility. The benchmark index is the Artificial Analysis Intelligence Index because it covers all 106 models in the estimation sample. The choice of intelligence measure does

Table 12: Prompt interactions for the accuracy bins (Γ) and for the style features with the largest spreads (B^R), production estimate; Hessian standard errors in parentheses. The three columns are the PLS prompt scores.

Feature	$\times$ PLS 1	$\times$ PLS 2	$\times$ PLS 3
Accuracy: mid	3.269 (0.200)	-1.389 (0.356)	0.756 (0.316)
Accuracy: high	5.262 (0.162)	-0.430 (0.286)	-0.014 (0.263)
Length	1.017 (0.161)	-3.405 (0.296)	-1.820 (0.266)
Table	-1.257 (0.366)	-1.574 (0.618)	1.911 (0.606)
Emoji	-0.117 (0.214)	-0.170 (0.383)	-0.880 (0.375)
Assertive hedge	-0.607 (0.118)	0.496 (0.198)	0.241 (0.197)
Tentative hedge	-0.229 (0.167)	-0.081 (0.269)	-0.119 (0.263)
Positive tone	-0.169 (0.138)	-0.650 (0.226)	-0.055 (0.230)
Flesch ease	-0.173 (0.286)	0.353 (0.500)	-1.827 (0.599)

not drive these results. Re-estimating the full model with the Epoch Capabilities Index in place of the Artificial Analysis index, on the 82,419 battles between the 73 covered models, gives a second-stage coefficient of 0.243 logits per ten index points (standard error 0.044) and the same held-out fit per battle, 0.9345. Entering the Artificial Analysis index through two bins (0.159 and 0.356 logits for the middle and top bins, standard errors 0.077 and 0.098) or adding a quadratic term (neither term significant) leaves the fit within 0.002 per battle, and the logarithm of parameter count, available for every model, gives 0.102 logits per log unit (standard error 0.019). Across the six variants the accuracy gains move by less than 0.07 logits, from 1.07 to 1.14 for the middle bin and 1.70 to 1.82 for the top bin. The full comparison is in the replication package.

Plain logit fixed effects. The association between benchmark intelligence and the model fixed effects does not depend on the random coefficients. Table 13 repeats the second stage for two plain trinomial logits estimated on the same training battles: one with model fixed effects and the position term only, and one that adds the thirteen style features and the accuracy bins with common coefficients. In each case we regress the 73 named fixed effects on the Artificial Analysis index, the context window, and the reasoning indicator. A ten-point increase in the index is associated with 0.233 logits (standard error 0.036) when the fixed effects absorb everything, 0.123 (0.028) once style and accuracy are controlled, and 0.160 (0.030) in the mixed logit. The coefficient falls by about half when response characteristics enter, which is what one expects if part of what more capable models are rewarded for is more accurate and better formatted answers, and it is similar across the two specifications that control for them. The reasoning indicator is never distinguishable from zero. The negative context-window coefficient appears only in the mixed logit, where the context window also enters through prompt interactions and a random coefficient, which is a further reason not to interpret it.

Table 13: Second stage on the model fixed effects of three specifications estimated on the same training battles. OLS of the 73 named fixed effects on model characteristics; standard errors in parentheses.

	Logit, FE only	Logit, style and accuracy	Mixed logit
Constant	0.000 (0.142)	−0.805 (0.112)	−0.815 (0.121)
Intelligence index / 10	0.233 (0.036)	0.123 (0.028)	0.160 (0.030)
Context window (M tokens)	0.005 (0.028)	0.018 (0.022)	−0.183 (0.024)
Reasoning model	−0.130 (0.115)	−0.082 (0.090)	−0.070 (0.098)
R^2	0.53	0.33	0.60
Named models	73	73	73
Test log-likelihood	−12,776	−12,258	−12,079

A simple demand comparison. In a logit of whether model A wins a decided battle on A-minus-B differences (87,428 battles), the intelligence index alone is worth 0.2145 logits per ten points (HC1 standard error 0.005). Adding the context window, the reasoning indicator, the thirteen style differences, and the two accuracy-bin differences lowers the coefficient to 0.1660 (0.006), a 22.6% reduction, while the accuracy bins enter at 0.826 (0.022) and 1.161 (0.021) and McFadden’s R^2 rises from 0.017 to 0.070. Model fixed effects are excluded because they would absorb the time-invariant intelligence measure.

D Routing, learning, and steering details

D.1 Routing panel

The source panel begins with 5,000 topic-stratified prompts. We retain the 4,916 prompts with a response and complete features for all ten candidates. Gemini 2.5 Pro is both a candidate and the fixed baseline. The prompt router uses deterministic mean utility. The user-preference-level router adds a fixed simulated taste draw for each of 1,000 users. The same user population, seed, prompts, and candidate responses are used for both policies. Every panel response carries a judged accuracy score; the 20 responses that returned no text are scored zero, in the low bin.

The OpenRouter API identifiers used for the panel are:

GPT-4.1: openai/gpt-4.1; GPT-4o: openai/gpt-4o-2024-11-20; Claude Sonnet 4: anthropic/claude-sonnet-4; Gemini 2.5 Pro: google/gemini-2.5-pro; DeepSeek V3: deepseek/deepseek-chat-v3-0324; Llama 4 Maverick: meta-llama/llama-4-maverick; Grok 4: x-ai/grok-4; Qwen3 235B: qwen/qwen3-235b-a22b-2507; Kimi K2: moonshotai/kimi-k2; and MiniMax M1: minimax/minimax-m1.

Responses were generated in May 2026 with a 1,500-token limit, which truncated about one response in five; we regenerated those in August and September 2026 for nine of the ten models under the panel’s final generation policy (temperature 0.7, a 4,000-token limit

doubled once if the response is still truncated, and a 128-token reasoning budget for Gemini 2.5 Pro so that the limit is spent on the answer). Thirteen MiniMax M1 cells remain at the original limit because the model returns no text when given a larger budget. Grok 4 had been retired by then, so its responses are frozen as first generated and 914 of them (18.6%) end at the limit. The preference model does not observe whether a response is finished, so this could bias Grok’s utility either way. It does not appear to: on the 131 prompts of similar length where Grok’s answer is complete, its gap to Gemini 2.5 Pro is 0.64 logits against 0.83 on the truncated prompts, and imputing the complete-answer gap changes the prompt-router gain by 0.005 logits and the gain of the user-preference-level router by 0.004.

Population shares and topic leaders. When each of 1,000 simulated users picks their utility-maximizing model on each prompt, Gemini 2.5 Pro receives 17.4% of picks (prompt-clustered standard error 0.11), Grok 4 13.4%, Qwen3 235B 11.7%, DeepSeek V3 10.7%, GPT-4.1 10.1%, MiniMax M1 8.8%, Claude Sonnet 4 8.2%, Kimi K2 7.4%, GPT-4o 6.4%, and Llama 4 Maverick 5.9%. The gap between the top two is 4.0 percentage points ($t = 28.6$). Gemini 2.5 Pro leads information seeking, practical guidance, writing, coding, and casual conversation, Qwen3 235B leads mathematics, and Grok 4 leads multimedia prompts; the leader is distinguishable from the runner-up at the 5% level in every topic except multimedia.

D.2 Online learning

For each of 200 simulated users, round t compares the model with the largest posterior mean utility to a uniformly random challenger. A/B position is randomized. The user’s A, B, or outside choice is drawn from the trinomial logit under their fixed true taste. After each response we maximize the log posterior, consisting of accumulated choice log-likelihood plus the standard-normal log prior on ν_i ; all eighteen taste components, including the two accuracy tastes, are learned. Evaluation uses 800 prompts not used for feedback. User-level intervals in Figure 2 are computed from the cross-user sampling distribution at each checkpoint.

Table 14: Learning curve. Gap closed is relative to the prompt-router prior and the user-preference-level router on this 200-user, 800-prompt sample.

	Rounds	0	1	5	10	20	30	40
Uplift vs. Gemini 2.5 Pro		0.327	0.412	0.646	0.760	0.913	0.997	1.061
Implied win rate		54.3%	55.5%	59.2%	61.0%	63.4%	64.9%	66.0%
Gap closed		0%	8.4%	31.3%	42.5%	57.6%	65.8%	72.1%

D.3 Verbal personalization

The 100 taste profiles are drawn from the production estimate: from a pool of 20,000 draws of $\nu \sim N(0, I_{18})$ we stratify the implied length coefficient into 100 quantile bins and keep the draw nearest each bin center, so the profiles span the distribution of length taste while carrying that draw’s tastes for every other feature, including the two accuracy bins. The verbalizer ranks a profile’s style deviations $\sigma^R \odot \nu_i^R$ by absolute magnitude, applies a threshold of 0.6 utility units, and emits at most four instructions. All 100 profiles receive a nonempty preamble. Each model is queried through OpenRouter under the routing panel’s generation policy (temperature 0.7, a 4,000-token limit doubled once on truncation, and a 128-token reasoning budget for Gemini 2.5 Pro) on 100 topic-stratified prompts; truncated or empty generations are retried and, if still defective, excluded, leaving between 9,619 (MiniMax M1) and 10,000 scored cells per model. Every steered and base response is scored by the accuracy judge. The utility difference is the intended user’s prompt-specific style coefficients applied to the feature differences plus the user’s own prompt-specific accuracy gain applied to the difference in accuracy bins. No model fixed effect enters because both responses come from the same model.

The prompt-clustered standard error treats the 100 prompts as independent clusters and allows arbitrary dependence across the 100 profiles within a prompt. The cross-matrix evaluates every response created for profile j under all profiles i . For Gemini 2.5 Pro its 100 diagonal profile assignments average 1.448; its 9,900 off-diagonal assignments average -0.014 . Under the average user’s tastes ($\nu = 0$) the steered responses are worth between -0.11 (DeepSeek V3) and $+0.09$ (Claude Sonnet 4) logits across the nine models.

The comparison with routing in Table 6 uses the 100 verbal prompts, which are a subset of the routing panel, and the same 100 profiles as users. Each policy is scored against Gemini 2.5 Pro’s unsteered panel response under the user’s full taste, including model fixed effects, on the 9,745 user-prompt cells in which every steered response exists. The nine-model routing rows restrict the menu to the models that can be steered; the ten-model rows add the frozen Grok 4 panel response. Fresh unsteered generations differ from the panel response for the same prompt by -0.09 to $+0.27$ logits depending on the model, which is the sampling noise of Section 7 and not a steering effect.

E Ex-ante router details

The routers of Section 6 run on the 4,916-prompt panel with 4,000 simulated users and seed 2026. Table 15 lists every predictor. A1’s maximum-a-posteriori taste estimate uses the same trinomial likelihood and standard-normal prior as the online learner of Section 4.3, applied to k passively observed choices on learning-split prompts. A2 is a multinomial logistic regression from a 21-dimensional summary of the choice history (how often each

model was offered, how often each was chosen, and how often the user rejected both) to the identity of the user’s A0-optimal model; its label accuracy on held-out users rises from 0.44 at $k = 0$ (the modal model) to 0.57 at $k = 40$. B1 and B3 are ridge regressions with per-target penalties chosen by cross-validation; B2 and B4 are 3072-512-256 networks trained with AdamW and early stopping. The routing R^2 in the table is computed after removing each model’s mean over prompts and each prompt’s mean over models, and is the only fit statistic that bears on a pick.

Table 15: Predictors estimated for the ex-ante routers. Out-of-fold R^2 on utility and on the routing-relevant prompt-by-model interaction; taste recovery is the mean correlation between the estimated and true taste vectors.

Variant	Estimator	Parameters	Fit
A1 ($k = 5/10/20/40$)	MAP posterior of ν_i	18 per user	taste recovery 0.23 / 0.31 / 0.38 / 0.47
A2 ($k = 5/10/20/40$)	multinomial logistic to model id	198	label accuracy 0.44 / 0.48 / 0.52 / 0.57
B1	ridge, embedding to utility (10 targets)	30,720	R^2 0.254, routing 0.113
B2	network, embedding to utility	1,707,274	R^2 0.293, routing 0.110
B3	ridge, embedding to 15 characteristics $\times$ 10 models	460,800	R^2 0.276 (characteristics), 0.298 (utility), routing 0.135
B4	network, embedding to characteristics	1,743,254	R^2 0.305 (characteristics), 0.304 (utility), routing 0.117

The full grid also sweeps $k \in \{0, 1, 5, 10, 20, 40\}$ for A1, A2, and C2. At $k = 0$ every learned rule collapses to the fixed baseline; at $k = 1$ the gains are 0.008 (A1), 0.001 (A2), and 0.063 (C2) logits; at $k = 5$, 0.093, 0.033, and 0.186; at $k = 20$, 0.245, 0.162, and 0.374. The number of distinct models a rule uses grows with k : A1 spreads users over 6 models from $k = 5$ on and A2 over 9 at $k = 40$, while C2 uses all ten from $k = 1$ onward because the prompt-side prediction varies the pick within a user.

F Sampling-noise audit details

The replicated grid holds 497 topic-stratified prompts drawn from the routing panel (70 to 72 per topic), the nine models that can still be queried, and five fresh OpenRouter draws per cell under the panel’s generation policy. Every replicate is judged for accuracy and scored with the production model’s deterministic-mean utility. The two-way ANOVA with interaction rejects the null of no model effect ($F = 2,084$) and of no model-by-prompt interaction tested against replicate noise ($F = 7.8$, $p < 10^{-30}$); the structural terms explain 84% of the variance and within-cell resampling the remainder. Per model, the within-cell standard deviation ranges from 0.32 logits (Claude Sonnet 4 and Llama 4 Maverick) through 0.40 (Gemini 2.5 Pro) to 0.55 (MiniMax M1). By topic, the within-cell share of variance runs from 13% (writing) to 31% (multimedia) and the interaction share from

17% (practical guidance) to 34% (mathematics). The variance shares carry prompt-cluster bootstrap 95% intervals of [14.9, 18.8] for model, [32.2, 40.4] for prompt, [24.4, 30.0] for the interaction, and [17.8, 22.1] for within-cell resampling.

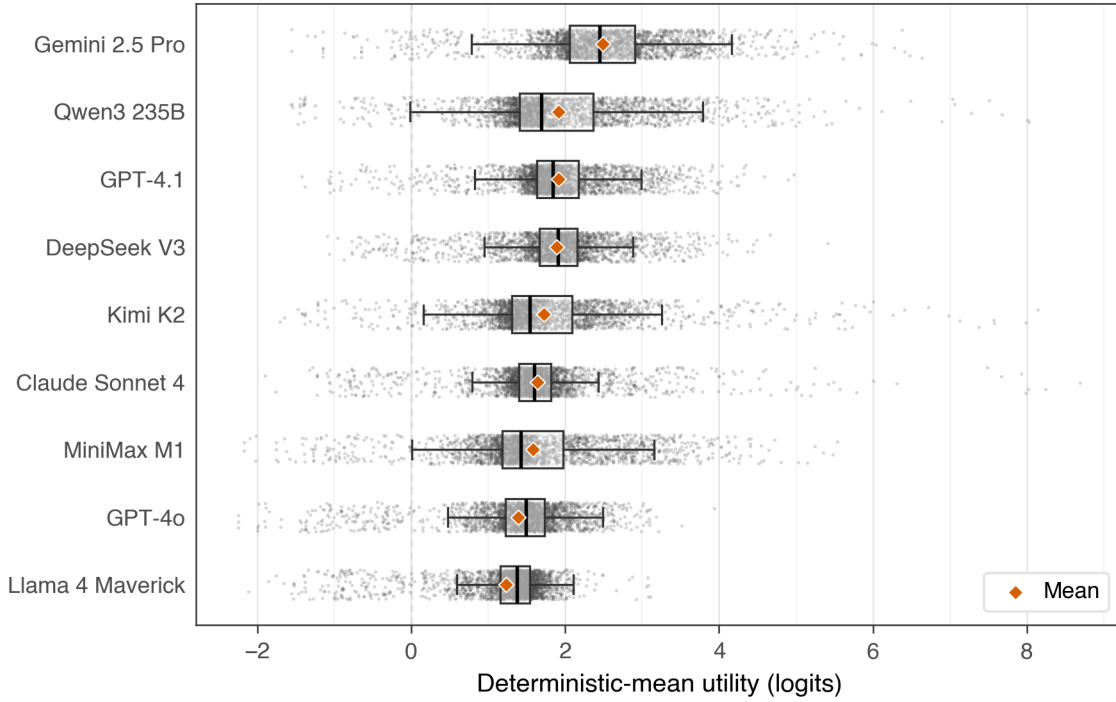


Figure 6: Deterministic-mean utility of every replicate in the sampling audit, by model, ordered by mean. Points are individual re-draws, roughly 2,500 per model across the 497 prompts. Boxes span the interquartile range with the median marked; whiskers extend to 1.5 times the interquartile range; diamonds mark means.

The preference-learning mechanism can also create a feedback loop: models receive traffic where the router already believes they perform well, generating selective data that may reinforce initial estimates. A deployment would need randomized exploration and monitoring by prompt group to distinguish genuine specialization from a self-confirming allocation.